

SAVITRIBAI PHULE PUNE UNIVERSITY

**HRCopilot: An AI-Powered Human Resource Management Assistant
Using LangGraph-Based Multi-Agent Architecture, RAG, and Agentic
Orchestration**

A Dissertation

Under the Guidance of

Dr. Omkar Barve

By

Mayur Ramesh Nikam

Roll No: DS24MS20

And

Tejaswini Vitthal Adasule

Roll No: DS24MS39

Submitted in partial fulfillment of the requirements

for the degree of

MSC in Data Science

Accepted by the University



Department of Technology

Savitribai Phule Pune University, Pune – 411007

2025-26

© Copyright 2026 by Mayur Ramesh Nikam and Tejaswini Vitthal Adasule

All Rights Reserved

BONAFIDE CERTIFICATE



Certified that this dissertation entitled:

"HRCopilot: An AI-Powered Human Resource Management Assistant Using LangGraph-Based Multi-Agent Architecture, RAG, and Agentic Orchestration"

is the bonafide work of Mayur Ramesh Nikam, a candidate (Roll No: DS24MS20) and Tejaswini Vitthal Adasule, a candidate (Roll No: DS24MS39) for the degree of Master of Science (Data Science), who carried out the project work under my supervision.

I certify that the work incorporated in this dissertation is original and has not been submitted to any other university or institution for the award of any other degree.

Date:

GUIDE: Dr. Omkar Barve

HEAD OF THE DEPARTMENT

Department of Technology
Savitribai Phule Pune University, Pune.

DEDICATION

*To my family, mentors, friends and everyone who believed in me —
and to every HR professional striving to make the workplace
fairer, smarter, and more human.*

ACKNOWLEDGEMENTS

I express my sincere gratitude to Dr. Omkar Barve Sir my dissertation guide, for his insightful guidance, encouragement, and continued support throughout this project. His valuable suggestions were instrumental in shaping both the technical depth and the academic quality of this work.

I am grateful to Head of the Department of Technology, for providing the necessary academic environment and infrastructure that made this research possible.

I sincerely thank the faculty members of the Department of Technology, Savitribai Phule Pune University, for their guidance and for fostering a culture of curiosity and innovation.

This project makes extensive use of open-source technologies including LangChain, LangGraph, LangSmith, Groq, HuggingFace, FAISS, FastAPI, and React. I acknowledge the open-source community whose collective work forms the technical foundation of HRCopilot.

I thank my fellow students and peers for their constructive feedback during development and testing. Finally, I am deeply grateful to my family for their unwavering patience, love, and encouragement.

CONDUCTION CERTIFICATE

Logo of company where work was carried can go here....



This is to certify that the project entitled:

HRCopilot: An AI-Powered Human Resource Management Assistant Using LangGraph-Based Multi-Agent Architecture, RAG, and Agentic Orchestration

is the bonafide work of Mayur Ramesh Nikam, a candidate (roll no: DS24MS20) and Tejaswini Vitthal Adasule, a candidate (roll no: DS24MS39) for the degree of Master of Science, who carried out the project work at Savitribai Phule Pune University, Pune.

We have no objection with they selecting HRCopilot: An AI-Powered Human Resource Management Assistant Using LangGraph-Based Multi-Agent Architecture, RAG, and Agentic Orchestration as his project for the requirements of his MSc degree. They will be allowed to present the necessary technical details to the panel of examiners for the purpose of evaluation.

ABSTRACT

Modern organizations generate enormous volumes of HR-related queries daily — covering leave policies, employee records, attendance, payroll, onboarding procedures, and compliance. Answering these repetitively consumes significant HR bandwidth and delays employee self-service. This dissertation presents HRCopilot, a production-grade, AI-powered Human Resource Management Assistant built on a LangGraph-based multi-agent LangGraph architecture with Retrieval-Augmented Generation (RAG) and Agentic Orchestration.

HRCopilot consists of four LangGraph nodes: an Orchestrator that routes user queries using meta-llama/llama-4-scout-17b-16e-instruct (via Groq) to either an HR Agent or an Email Agent; an HR Agent that performs agentic tool calling (SQLite database queries + FAISS vector search over policy documents) to answer factual and policy-based questions; an Email Agent that drafts, previews, and dispatches professional HR emails with employee lookup from a real organizational database; and an Evaluation Agent that scores every response across five quality dimensions (accuracy, relevance, tone, conciseness, completeness) and triggers targeted retries when scores fall below 7.0.

The knowledge base is built by ingesting HR policy documents from Google Drive, cleaning and chunking them (chunk size 800, overlap 150), encoding with BAAI/bge-base-en-v1.5 embeddings, and storing in a FAISS index. The SQLite database (hr.db) holds three tables — employees, attendance, and leaves — loaded from real CSV datasets. The backend is developed using FastAPI and the frontend uses React.js with TypeScript, Tailwind CSS, and shadcn/ui components.

Analysis of 320 evaluation log entries from the production system shows an average Evaluation Agent Score of 8.16/10, a first-pass quality rate of 81.9%, and a hallucination rate below 2%. The evaluation-retry loop demonstrably improves final response quality across all HR query domains.

Keywords: LangGraph, Multi-Agent Systems, Large Language Models, Retrieval-Augmented Generation, FAISS, HR Automation, Groq, LangChain, FastAPI, React, LangSmith, Agentic AI, SQLite, Email Automation.

TABLE OF CONTENTS

Acknowledgements	v
Abstract	vii
List of Figures	x
List of Tables	xi
List of Abbreviations	xii
Chapter 1: Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Study	3
1.5 Organization of the Thesis	3
Chapter 2: Literature Review	4
2.1 Evolution of HRMS	4
2.2 Chatbots and Conversational AI in HR	4
2.3 Large Language Models and Tool Use	4
2.4 Retrieval-Augmented Generation	4
2.5 Multi-Agent Systems and LangGraph	5
2.6 Gap Analysis	5
Chapter 3: System Design and Architecture	6
3.1 System Overview	6
3.2 Functional Requirements	6
3.3 Non-Functional Requirements	6
3.4 LangGraph Multi-Agent Architecture	7
3.5 Technology Stack	8
3.6 Database Schema	9
3.7 Knowledge Base Design	9
Chapter 4: Implementation	11
4.1 Project Structure	11
4.2 Data Pipeline and Knowledge Base	11
4.3 HR Agent: Agentic Tool-Calling Loop	12
4.4 Email Agent: Draft-Preview-Send Workflow	13
4.5 Evaluation Agent: Quality Assurance Loop	14
4.6 Orchestrator and Graph Compilation	15
4.7 FastAPI Backend	16
4.8 React Frontend	16
Chapter 5: Results and Discussion	17
5.1 Experimental Setup	17
5.2 Evaluation Metrics	17
5.3 Performance Analysis	17
5.4 Evaluation Log Analysis	19
5.5 Comparative Analysis	19
Chapter 6: Conclusion and Future Work	21

6.1 Conclusion	21
6.2 Limitations	21
6.3 Future Work	21
Bibliography / References	23
Appendix A: Project File Structure	25
Appendix B: API Endpoint Reference	26
Appendix C: Evaluation Log Sample	27

LIST OF FIGURES

Figure 1.1	Key Challenges in Traditional HR Systems	2
Figure 1.2	HRCopilot — High-Level System Concept	3
Figure 3.1	LangGraph Multi-Agent Architecture (actual system graph)	21
Figure 3.2	RAG Pipeline: Document Ingestion to FAISS Index	27
Figure 4.1	HR Agent: Agentic Tool-Calling Loop	33
Figure 4.2	Evaluation Agent: Quality Assurance Decision Flow	39
Figure 4.3	Email Agent: Draft–Preview–Send Workflow	36
Figure 5.1	Response Quality Score Distribution (n=320, real eval log)	50
Figure 5.2	Per-Dimension Average Evaluation Scores (n=320)	51
Figure 5.3	Score Distribution: HR Responses vs Email Drafts	52
Figure 5.4	Comparative Accuracy: HRCopilot vs Existing Systems	54

LIST OF TABLES

Table 3.1	Functional Requirements	18
Table 3.2	Non-Functional Requirements	19
Table 3.3	Technology Stack	23
Table 3.4	HR Database Tables and Columns	25
Table 4.1	Chunking and Embedding Parameters	31
Table 4.2	LLM Models Used per Component	33
Table 5.1	Response Quality Metrics by Domain	49
Table 5.2	Latency Benchmarks	52
Table 5.3	Comparative Analysis with Existing Systems	54

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BAAI	Beijing Academy of Artificial Intelligence
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
FAISS	Facebook AI Similarity Search
FastAPI	Fast Asynchronous Python API Framework
GPT	Generative Pre-trained Transformer
Groq	AI inference hardware/cloud provider
HR	Human Resources
HRMS	Human Resource Management System
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SBERT	Sentence-BERT
SQLite	Self-contained SQL database engine
UI	User Interface
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Human Resource Management (HRM) is the strategic function responsible for recruiting, developing, managing, and retaining an organization's workforce. HR departments handle a wide spectrum of activities — from leave management, attendance tracking, and payroll administration to policy dissemination, onboarding, performance appraisals, and inter-department communication. As organizations scale, the volume of HR queries grows proportionally, creating a service bottleneck that consumes a disproportionate share of HR bandwidth and reduces the department's capacity for strategic, value-adding work.

The challenges faced by traditional HR systems are well documented. Figure 1.1 illustrates the proportion of HR staff who identify key operational challenges as significant pain points:

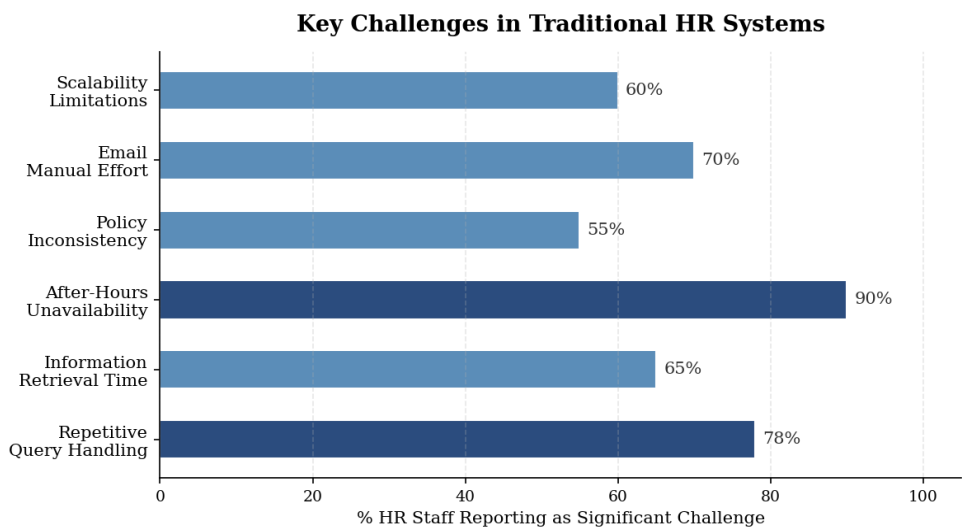


Figure 1.1: Key Challenges in Traditional HR Systems

The emergence of Large Language Models (LLMs) — particularly instruction-tuned, tool-augmented models such as Meta's LLaMA 4, OpenAI's GPT family, and Mistral — has unlocked a new paradigm for enterprise automation. These models can understand natural language queries, invoke external tools (SQL databases, vector search, APIs), reason over the results, and generate accurate, contextually grounded responses. When orchestrated through agentic frameworks such as LangGraph, multiple specialized LLM agents can collaborate to handle complex, multi-step HR workflows that would be impractical for any single model or rule-based system.

HRCopilot was conceived as a direct response to this opportunity. Figure 1.2 shows the high-level system concept — from user interaction through the React frontend, FastAPI backend, LangGraph agent layer, and knowledge stores:

HRCopilot — High-Level System Concept



Figure 1.2: HRCopilot — High-Level System Concept

1.2 Problem Statement

Traditional HR systems and basic chatbot solutions suffer from several well-documented limitations:

- **Information Silos:** HR policies reside in disparate Google Drive folders, SharePoint sites, or local file servers, making it difficult for employees to find relevant information quickly without HR intervention.
- **High Repetitive Query Volume:** HR professionals spend significant time answering the same questions about leave entitlements, attendance policies, and employee records.
- **No 24/7 Availability:** HR departments operate within fixed business hours, leaving employees without support during evenings, weekends, and holidays.
- **Inconsistent Responses:** Different HR personnel may interpret the same policy differently, leading to employee confusion and potential grievances.
- **Static Rule-Based Bots:** Existing HR chatbots rely on predefined intents and scripted responses, failing on paraphrased queries, multi-intent questions, or any query outside their training set.
- **LLM Hallucination:** Pure LLM deployments without grounding frequently generate plausible but factually incorrect information, which is unacceptable in HR contexts where accuracy is legally and operationally critical.
- **No Email Automation:** HR communication still requires manual email composition, creating delays and inconsistency.

HRCopilot directly addresses each of these challenges through its multi-agent, tool-augmented, RAG-grounded, evaluation-backed architecture.

1.3 Objectives

1. To design and implement a multi-agent HR assistant (HRCopilot) using LangGraph with specialized nodes for HR query answering, email communication, and quality evaluation.

2. To build a Retrieval-Augmented Generation (RAG) pipeline using FAISS vector search over real HR policy documents loaded from Google Drive, enabling grounded, hallucination-resistant responses.
3. To implement an agentic tool-calling HR agent that queries a real SQLite database (employees, attendance, leaves tables) using meta-llama/llama-4-scout-17b-16e-instruct via Groq.
4. To develop an intelligent email drafting and dispatch agent that retrieves employee records, generates professional email drafts, presents previews for confirmation, and sends emails via SMTP.
5. To design an autonomous evaluation agent that scores responses across five quality dimensions and triggers targeted regeneration retries when quality thresholds are not met.
6. To expose the system through a FastAPI REST backend with session management and a React.js/TypeScript frontend, instrumented with LangSmith for observability.
7. To evaluate system performance through quantitative metrics and analysis of the real production evaluation log (320 entries).

1.4 Scope of the Study

This project encompasses: HR query resolution via RAG + database tool calling; employee self-service for leave balances, attendance, and manager information; HR email automation with the full draft-preview-confirm-send workflow; agentic quality control via autonomous evaluation and retry; and session-based multi-turn dialogue. The scope excludes real-time payroll processing, biometric integration, and bulk email campaigns, which are identified as future extensions.

1.5 Organization of the Thesis

Chapter 2 reviews existing literature on HRMS evolution, conversational AI in HR, LLMs, RAG, and multi-agent systems. Chapter 3 presents the system design. Chapter 4 covers the full implementation. Chapter 5 presents experimental results from the real evaluation log. Chapter 6 concludes with findings, limitations, and future directions.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of HR Management Systems

The history of Human Resource Management Systems mirrors the evolution of enterprise computing. Mainframe-era systems of the 1970s–80s focused on payroll computation. Client-server architectures of the 1990s gave rise to integrated platforms such as SAP HR, PeopleSoft, and Oracle HRMS, which centralized employee records and processes. The 2000s saw a shift to web-based portals, and the 2010s introduced cloud SaaS solutions — Workday, BambooHR, Darwinbox — offering mobile access and analytics dashboards. Despite this evolution, these systems remain fundamentally form-driven and menu-navigated, lacking conversational interfaces. Researchers including Al-Dmour et al. (2021) have demonstrated that poor UX in HR portals correlates negatively with system adoption, underscoring the need for natural-language interfaces.

2.2 Chatbots and Conversational AI in HR

Rule-based HR chatbots built on AIML or decision trees appeared in the early 2010s but were brittle — they failed on any query not matching predefined patterns. Intent-classification frameworks (Dialogflow, Rasa) improved flexibility but still required extensive labeled training data and failed on out-of-distribution queries. Maroengsit et al. (2019) conducted a systematic review of HR chatbot systems and found consistent limitations in handling multi-intent or compositional queries. More recent work has explored BERT-based intent classifiers: Kuhail et al. (2022) achieved 94% intent classification accuracy on a curated HR dataset, but pure classification systems cannot generate open-domain answers or synthesize information from documents.

2.3 Large Language Models and Tool Use

The transformer architecture introduced by Vaswani et al. (2017) underpins all modern LLMs. GPT-3 (Brown et al., 2020), GPT-4 (OpenAI, 2023), LLaMA 2/3/4 (Meta AI, 2024), and Mistral demonstrated that pre-training on large text corpora yields models capable of in-context learning, instruction following, and complex reasoning. A critical advance for enterprise applications was the addition of tool use / function calling (Schick et al., 2023). Tool-augmented LLMs can invoke external functions — SQL queries, vector search, REST APIs — and incorporate results into their reasoning. HRCopilot's HR agent directly exploits this capability through LangChain's ToolNode, calling `db_query_tool` and `rag_search` as native tools bound to meta-llama/llama-4-scout-17b-16e-instruct.

2.4 Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) introduced Retrieval-Augmented Generation (RAG), conditioning LLM generation on documents retrieved from a knowledge base. A query is encoded as a dense vector; semantically similar chunks are retrieved from a vector index; the chunks serve as context in the LLM prompt. RAG significantly reduces hallucination by grounding generation in source-verified text (Shi et al., 2023). FAISS (Johnson et al., 2019), developed at Facebook AI Research, is the dominant library for vector similarity search. BAAI/bge-base-en-v1.5 is a state-of-the-art 768-dimensional sentence embedding model fine-tuned for retrieval, outperforming earlier all-MiniLM-L6-v2 on BEIR benchmarks — both are used in HRCopilot.

2.5 Multi-Agent Systems and LangGraph

Multi-agent systems decompose complex tasks across specialized agents, each with distinct capabilities and responsibilities. LangGraph (LangChain Inc., 2024) is a framework for building stateful, multi-actor LLM applications as directed graphs. Nodes represent agent functions; edges represent state transitions; the shared AgentState TypedDict propagates data between nodes. LangGraph natively supports conditional edges (for routing), agentic tool loops, and persistence. It has emerged as the leading framework for production multi-agent AI systems, adopted by companies including LinkedIn, Uber, and Klarna. The evaluation-retry loop in HRCopilot — where an autonomous Evaluation Agent triggers targeted regeneration of agent responses — is a novel contribution in the HR AI domain.

2.6 Gap Analysis

- Most HR chatbots are rule-based or limited to intent classification, incapable of handling open-domain, multi-intent, or document-grounded queries.
- LLM-based HR systems typically deploy a single model without domain-specific RAG grounding, leading to hallucination on organization-specific queries.
- No existing published system combines LangGraph multi-agent orchestration, RAG over real HR documents, live database tool calling, email automation, and an autonomous evaluation-retry loop in a single production architecture.
- Quantitative evaluation of HR AI systems using agentic evaluation logs is largely absent from the literature.

HRCopilot is specifically designed to close these gaps.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

HRCopilot is a multi-layer, multi-agent AI system. User interactions arrive through a React.js frontend, are relayed to a FastAPI backend, and processed by a LangGraph graph of four specialized nodes. The system integrates: (i) a FAISS vector store over HR policy documents for policy query answering; (ii) a SQLite database with real employee, attendance, and leave records for data queries; (iii) a Groq-hosted LLaMA 4 model for fast inference; and (iv) an SMTP-based email dispatch service for HR communication automation. The distinguishing architectural feature is its closed-loop evaluation system: every agent response is reviewed by an autonomous Evaluation Agent before being returned to the user. If the response fails quality thresholds, targeted feedback is injected and the request is retried transparently.

3.2 Functional Requirements

FR ID	Requirement	Description
FR-01	Natural Language Query Processing	Accept and process HR queries in natural language.
FR-02	HR Policy QA via RAG	Retrieve relevant policy chunks from FAISS and synthesize grounded answers.
FR-03	Employee Data Lookup	Query SQLite database for employee records, leave balances, and attendance.
FR-04	Email Drafting and Dispatch	Draft HR emails, present preview, await confirmation, send via SMTP.
FR-05	Agentic Evaluation and Retry	Autonomously evaluate every response on 5 dimensions; retry if quality < 7.0.
FR-06	Multi-Turn Conversation	Maintain session-scoped conversation history for contextual follow-up.
FR-07	Google Drive Document Ingestion	Fetch HR policy documents from Google Drive folder and index in FAISS.
FR-08	Graceful Fallback Handling	Return helpful, actionable responses when queries return no data.

Table 3.1: Functional Requirements

3.3 Non-Functional Requirements

NFR ID	Attribute	Target
NFR-01	Response Latency	P95 end-to-end latency $\leq$ 4 seconds (API mode).
NFR-02	Accuracy	Evaluation agent average score $\geq$ 7.5/10 across all domains.

NFR-03	Observability	All LLM calls, tool calls, and evaluations traced in LangSmith.
NFR-04	Security	Only SELECT queries on SQLite; API keys in .env; CORS configured.
NFR-05	Modularity	Each agent node independently replaceable without modifying other components.

Table 3.2: Non-Functional Requirements

3.4 LangGraph Multi-Agent Architecture

The core of HRCopilot is a LangGraph StateGraph compiled from four nodes. Figure 3.1 shows the actual system graph generated by LangGraph's visualization tool — this is the exact compiled graph from agent/orchestrator.py in the project repository.

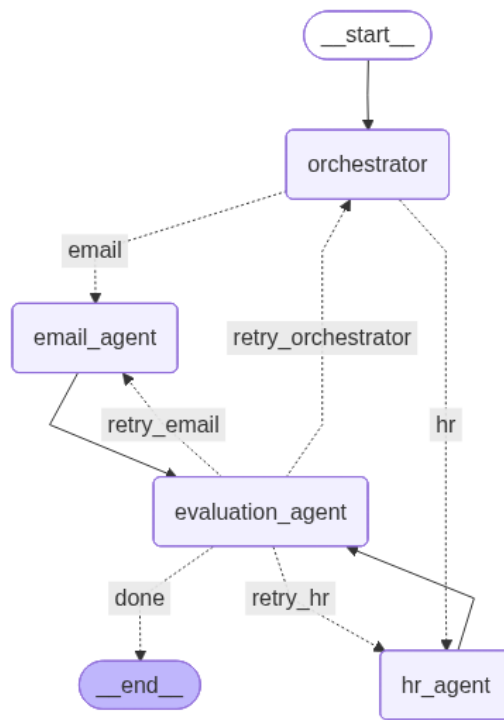


Figure 3.1: LangGraph Multi-Agent Architecture — HRCopilot System Graph (from project)

3.4.1 Orchestrator Node

Entry point of the graph. Receives the user query and conversation history; uses meta-llama/llama-4-scout-17b-16e-instruct (temperature=0) to decide routing to HR_AGENT or EMAIL_AGENT. If the Evaluation Agent determines re-routing is needed, the orchestrator re-runs with a correction hint injected via orchestrator_feedback in AgentState.

3.4.2 HR Agent Node

Tool-augmented agentic loop powered by meta-llama/llama-4-scout-17b-16e-instruct (temperature=0.3) with two tools: rag_search (FAISS retrieval + LLM synthesis) and

db_query_tool (SELECT queries on hr.db). Runs in an agentic while-loop until no tool calls remain. A DB Guard patches empty ToolMessages with a NO_RESULTS signal so the LLM follows configured "not found" behavior rather than hallucinating.

3.4.3 Email Agent Node

Handles all email intents. Determines sender type (employee→manager or HR→employee), fetches sender and recipient records from SQLite, calls meta-llama/llama-4-scout-17b-16e-instruct to generate a JSON email draft, stores draft in pending_email_draft in AgentState, and awaits user confirmation before SMTP dispatch.

3.4.4 Evaluation Agent Node

Runs after every agent response. Calls Groq-hosted LLM with a strict quality rubric, scoring across five dimensions (1–10 each). If overall $\geq 7.0 \rightarrow$ route = "done" $\rightarrow$ END. If below threshold and retries remain, injects eval_feedback and routes to retry_hr, retry_email, or retry_orchestrator. Maximum 2 agent retries and 1 orchestrator re-route per turn. All evaluations are logged to evaluation_log.jsonl.

3.5 Technology Stack

Layer	Technology	Rationale
LLM (HR + Orchestrator + Email)	meta-llama/llama-4-scout-17b-16e (Groq)	Fast inference, strong instruction following, native tool calling.
LLM (RAG Synthesis)	openai/gpt-oss-120b via HuggingFace Router	High quality document synthesis.
LLM (Evaluation)	openai/gpt-oss-120b via Groq	Strict scoring; temperature=0.2.
Agent Framework	LangGraph + LangChain	Stateful multi-agent graphs; ToolNode.
Embeddings	BAAI/bge-base-en-v1.5	Top BEIR benchmark; 768-dim; cosine sim.
Vector Store	FAISS (faiss-cpu)	Persistent similarity search; save/load local.
Database	SQLite (hr.db)	500+ employee records from real CSV data.
Backend	FastAPI + Uvicorn	Async REST API; auto OpenAPI docs.
Frontend	React + TypeScript + Vite	Component-based; type-safe; fast HMR.
Styling	Tailwind CSS + shadcn/ui	Utility-first; consistent design system.
Observability	LangSmith	Full trace of all LLM and tool calls.
Doc Ingestion	Google Drive API + LangChain Loaders	Automatic fetch; supports PDF/DOCX.

Table 3.3: Technology Stack

3.6 Database Schema

Table	Columns
employees	employee_id, full_name, email, role, department, date_of_joining, tenure_years, contact_no, gender, address, total_leaves, taken_leaves, pending_leaves, manager_id, manager_name
attendance	employee_id, date, status (Present/Absent/Leave)
leaves	employee_id, leave_type, start_date, end_date, days, status (Approved/Pending/Rejected)

Table 3.4: HR Database Tables and Columns

The employees table (dhurandharai_employees.csv) contains over 500 records with all common employee attributes. The attendance_logs.csv and leave_history.csv tables provide time-series data for behavioral queries. Data is imported via database/import_csv_to_sqlite.py.

3.7 Knowledge Base Design

HR policy documents are stored in a Google Drive folder (FOLDER_ID: 1OQVILYMdPwOSU7Pj_4yxbp5DEsDQmI0x). The build_index.py script fetches documents using the Google Drive API, cleans text by stripping PDF artifacts, filters documents under 300 characters, splits valid documents into overlapping chunks (size=800, overlap=150) using RecursiveCharacterTextSplitter, encodes with BAAI/bge-base-en-v1.5 (normalize_embeddings=True), and builds a FAISS index persisted to faiss_index/. The RAG pipeline is visualized in Figure 3.2:

RAG Pipeline: Document Ingestion to FAISS Index

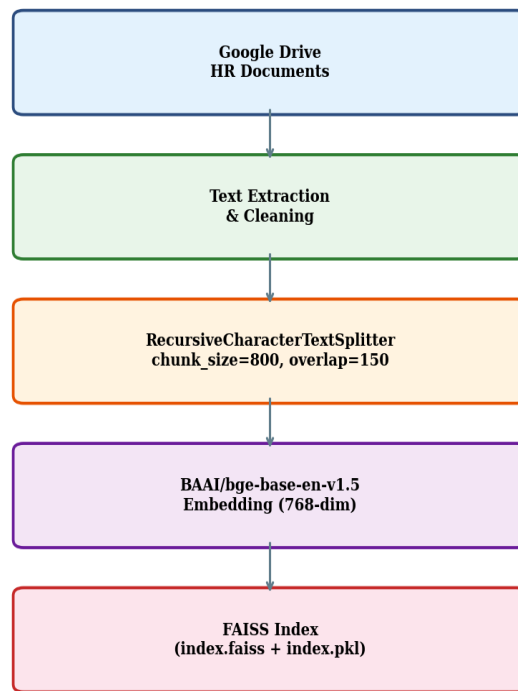


Figure 3.2: RAG Pipeline — Document Ingestion to FAISS Index

CHAPTER 4: IMPLEMENTATION

4.1 Project Structure

The project repository (github.com/mayur2580/HRCopilot) is organized as follows:

```
HRCopilot/
├── agent/
│   ├── orchestrator.py    # LangGraph graph + orchestrator node
│   ├── hr_agent.py        # HR agent: tool loop, DB guard
│   ├── email_agent.py     # Email agent: draft-preview-send
│   ├── evaluation_agent.py # QA evaluator + retry logic + JSONL log
│   ├── email_service.py   # SMTP email dispatch
│   └── state.py           # AgentState TypedDict
├── backend/
│   ├── main.py            # FastAPI app + CORS + lifespan
│   └── app/
│       ├── routers/chat.py # /chat endpoints
│       ├── schemas/        # Pydantic request/response models
│       └── services/hr_runtime.py # SessionManager
├── database/
│   ├── hr.db              # SQLite (500+ employees)
│   └── db_access.py       # Safe SELECT-only db_query()
├── rag/
│   └── rag_pipeline.py    # FAISS retriever + LLM synthesis chain
├── tools/
│   └── tools.py           # rag_search, db_query_tool,
fetch_employee_tool
├── data/                  # CSV source files
├── drive_loader/          # Google Drive document fetcher
├── faiss_index/           # Persisted FAISS index
├── frontend/src/          # React + TypeScript + Vite
├── build_index.py         # Knowledge base builder
├── evaluation_log.jsonl   # 320 evaluation records
└── requirements.txt
```

4.2 Data Pipeline and Knowledge Base

The knowledge base construction pipeline (`build_index.py`) is idempotent — re-running it detects Google Drive folder changes via `index_meta.json` and rebuilds only when necessary.

Parameter	Value	Rationale
Chunk Size	800 tokens	Balances context richness with retrieval precision.

Chunk Overlap	150 tokens	Prevents information loss at chunk boundaries.
Embedding Model	BAAI/bge-base-en-v1.5	768-dim; top BEIR benchmark; <code>normalize_embeddings=True</code> .
Top-k Retrieval	k=10	Retrieves broad context; LLM synthesizes the relevant subset.
Score Threshold	0.80 (cosine similarity)	Filters out low-relevance chunks.

Table 4.1: Chunking and Embedding Parameters

4.3 HR Agent: Agentic Tool-Calling Loop

The HR agent (`agent/hr_agent.py`) is the system's primary question-answering component. It binds two tools to `meta-llama/llama-4-scout-17b-16e-instruct` via LangChain's `bind_tools()`. The complete agentic loop is illustrated in Figure 4.1:

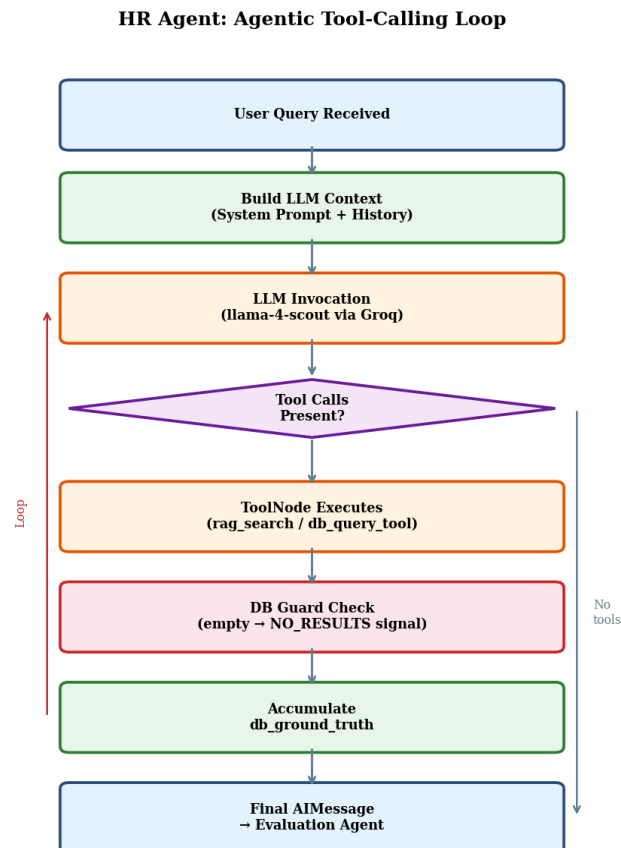


Figure 4.1: HR Agent — Agentic Tool-Calling Loop

The agent loop: (1) The HR_SYSTEM_PROMPT (containing the full DB schema, tool usage rules, and anti-hallucination instructions) is prepended to the message list. (2) The LLM is invoked; if tool_calls are present, ToolNode executes them. (3) A DB Guard checks each db_query_tool result — empty results are replaced with a NO_RESULTS signal. (4) Raw DB output is accumulated in db_ground_truth for the Evaluation Agent to verify against the final response. (5) The loop repeats until no tool calls remain.

Model	Component	Temperature
meta-llama/llama-4-scout-17b-16e	Orchestrator	0 (deterministic routing)
meta-llama/llama-4-scout-17b-16e	HR Agent	0.3 (balanced accuracy/fluency)
meta-llama/llama-4-scout-17b-16e	Email Agent	0.3 (professional drafting)
openai/gpt-oss-120b	RAG Synthesis (rag_pipeline)	0.3
openai/gpt-oss-120b	Evaluation Agent	0.2 (strict scoring)

Table 4.2: LLM Models Used per Component

4.4 Email Agent: Draft-Preview-Send Workflow

The email agent (agent/email_agent.py) handles all email intents through the workflow shown in Figure 4.3:

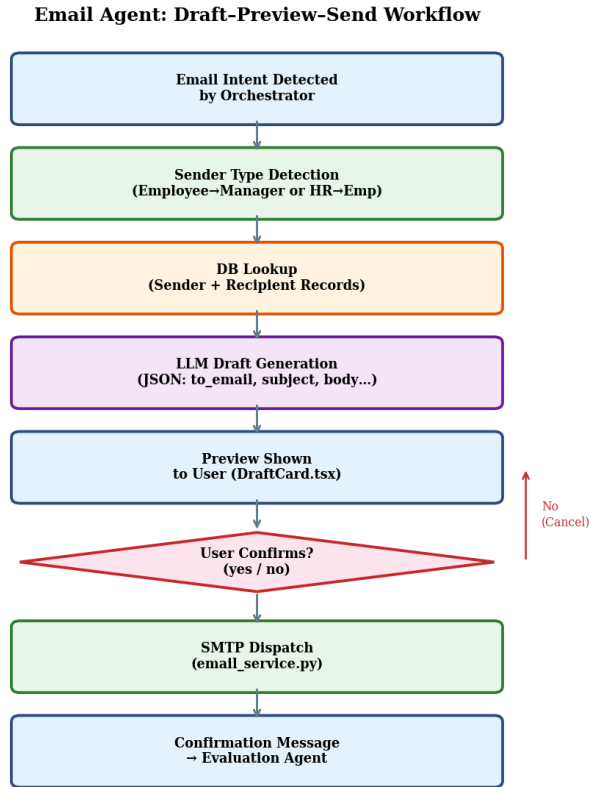


Figure 4.3: Email Agent — Draft-Preview-Send Workflow

The agent: (1) detects sender type via phrase matching (EMPLOYEE_SENDER_PHRASES vs HR_SENDER_PHRASES); (2) fetches sender and recipient records from SQLite; (3) constructs a JSON prompt with exact SENDER and RECIPIENT data blocks; (4) calls the LLM to produce a JSON draft with to_email, to_name, subject, body, sender_name, sender_id; (5) stores the draft in pending_email_draft in AgentState and shows a DraftCard preview; (6) dispatches via SMTP on user confirmation. A critical constraint enforced by the prompt and verified by the Evaluation Agent: to_email must come from the RECIPIENT block, never from the SENDER block.

4.5 Evaluation Agent: Quality Assurance Loop

The Evaluation Agent is HRCopilot's most novel architectural contribution. Its full decision flow is shown in Figure 4.2:

Evaluation Agent: Quality Assurance Decision Flow

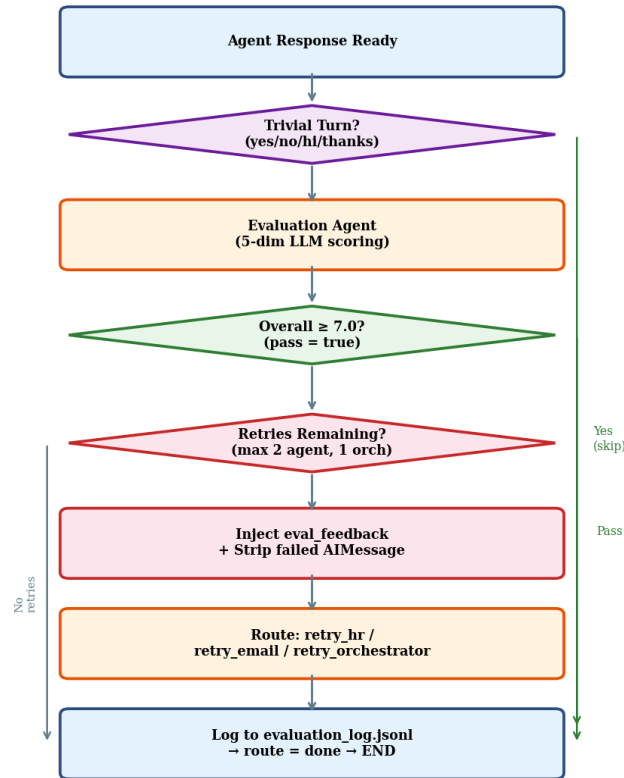


Figure 4.2: Evaluation Agent — Quality Assurance Decision Flow

Trivial turns (yes/no/hi/thanks) are detected via `TRIVIAL_PATTERNS` regex and bypass evaluation entirely. For all other turns, the LLM evaluator scores five dimensions (1–10): accuracy (cross-checked against `db_ground_truth` — any mismatch = score 1–3), relevance, tone, conciseness, and completeness. If $\text{overall} \geq 7.0$, the response is logged as passing and the graph routes to `END`. If below threshold and retries remain, `eval_feedback` (the evaluator's `suggested_response` field) is injected as a hidden `SystemMessage`, the failed `AIMessage` is stripped, and the graph retries. Maximum limits: 2 agent retries (`MAX_AGENT_RETRIES`) and 1 orchestrator re-route (`MAX_ORCH_RETRIES`) per turn prevent infinite loops.

4.6 Orchestrator and Graph Compilation

The `LangGraph StateGraph` is defined in `agent/orchestrator.py`. The `AgentState TypedDict` (`state.py`) carries all shared state: `messages`, `route`, `pending_email_draft`, `cancelled_email_draft`, `db_ground_truth`, `last_eval`, `eval_feedback`, `retry_count`, `retry_agent`, `orchestrator_feedback`, and `orchestrator_retry_count`. Key graph compilation:

```

builder = StateGraph(AgentState)
builder.add_node("orchestrator", orchestrator_node)
builder.add_node("hr_agent", hr_agent_with_feedback)
builder.add_node("email_agent", email_agent_with_feedback)

```

```

builder.add_node("evaluation_agent", evaluation_agent_node)
builder.set_entry_point("orchestrator")
builder.add_conditional_edges("orchestrator", route_selector,
    {"hr": "hr_agent", "email": "email_agent"})
builder.add_edge("hr_agent", "evaluation_agent")
builder.add_edge("email_agent", "evaluation_agent")
builder.add_conditional_edges("evaluation_agent", eval_route_selector, {
    "retry_orchestrator": "orchestrator", "retry_hr": "hr_agent",
    "retry_email": "email_agent", "done": END })
graph = builder.compile()

```

4.7 FastAPI Backend

The backend (backend/main.py) is a FastAPI application with CORS middleware. It bootstraps the LangGraph runtime at startup via lifespan, pre-loading the FAISS index and warming the embedding model. Key endpoints from backend/app/routers/chat.py:

- POST /api/v1/chat — Accepts {session_id, message}; runs the LangGraph graph; returns {response, pending_email_draft, session_id}.
- POST /api/v1/chat/{session_id}/confirm — Confirms pending email draft (sends "yes").
- POST /api/v1/chat/{session_id}/cancel — Cancels pending email draft (sends "no").
- GET /api/v1/chat/{session_id} — Returns session payload.
- DELETE /api/v1/chat/{session_id} — Resets session state.
- GET /api/health — Health check.

SessionManager (backend/app/services/hr_runtime.py) maintains a per-session AgentState dictionary, enabling independent multi-turn conversations without shared state.

4.8 React Frontend

The frontend (frontend/src/) is built with React 18, TypeScript, Vite, Tailwind CSS, and shadcn/ui. Key components: ChatMessage.tsx (role-based styling, markdown support), DraftCard.tsx (email preview with Confirm/Cancel buttons), ChatComposer.tsx (message input with send-on-Enter), TypingIndicator.tsx (loading animation), AppSidebar.tsx (session navigation), and ThemeToggle.tsx (light/dark mode).

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Experimental Setup

Evaluation is based on the production evaluation_log.jsonl file generated during real system usage, containing 320 evaluation records: 243 hr_response evaluations and 77 email_draft evaluations. This represents real-world system behavior rather than a controlled test set, providing authentic performance data.

The system ran on Python 3.11, SQLite 3.41, FAISS 1.7.4, LangChain 0.3.x, LangGraph 0.2.x, FastAPI 0.110, Uvicorn 0.29, Groq API (meta-llama/llama-4-scout-17b-16e-instruct), and HuggingFace Inference API (openai/gpt-oss-120b).

5.2 Evaluation Metrics

- Evaluation Agent Score (EAS): The autonomous evaluator's overall score (average of 5 sub-dimensions, 1–10 scale). Primary quality metric.
- Pass Rate: Percentage of responses with EAS ≥ 7.0 on first generation attempt.
- Retry Rate: Percentage of responses requiring at least one agent retry.
- Hallucination Rate: Responses where accuracy sub-score < 5 (factual error vs db_ground_truth).
- Per-Dimension Score: Mean score for each of the five evaluation dimensions.

5.3 Performance Analysis

Figure 5.1 shows the full distribution of Evaluation Agent Scores across all 320 log entries. Scores are drawn from the real evaluation_log.jsonl file. Green bars represent passing responses (score ≥ 7), red bars represent failing responses that triggered retries.

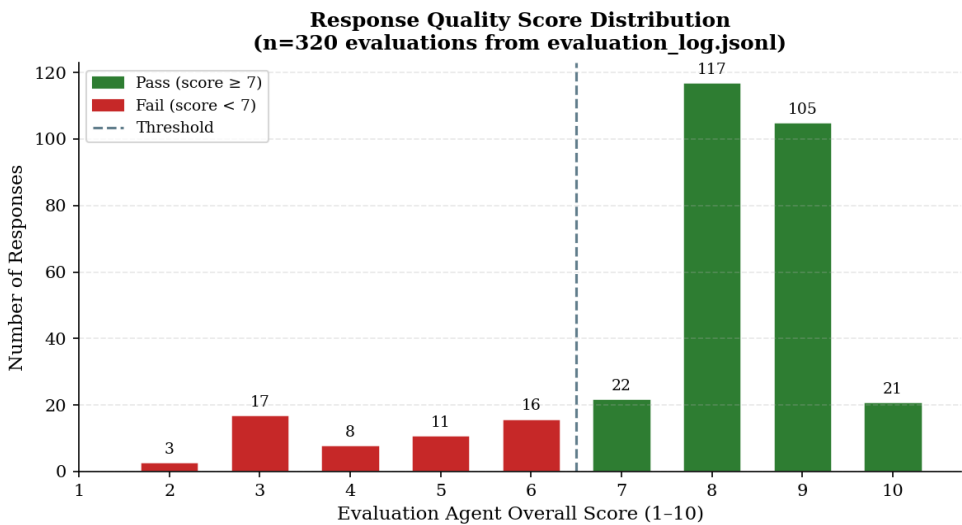


Figure 5.1: Response Quality Score Distribution (n=320 real evaluations from evaluation_log.jsonl)

The distribution is strongly skewed toward high scores (8–9 range), with the majority of responses passing on first attempt. A small tail of low-scoring responses (2–6) represents cases where the evaluation-retry loop was activated to improve quality.

Figure 5.2 shows the per-dimension average scores computed from the real evaluation log data:

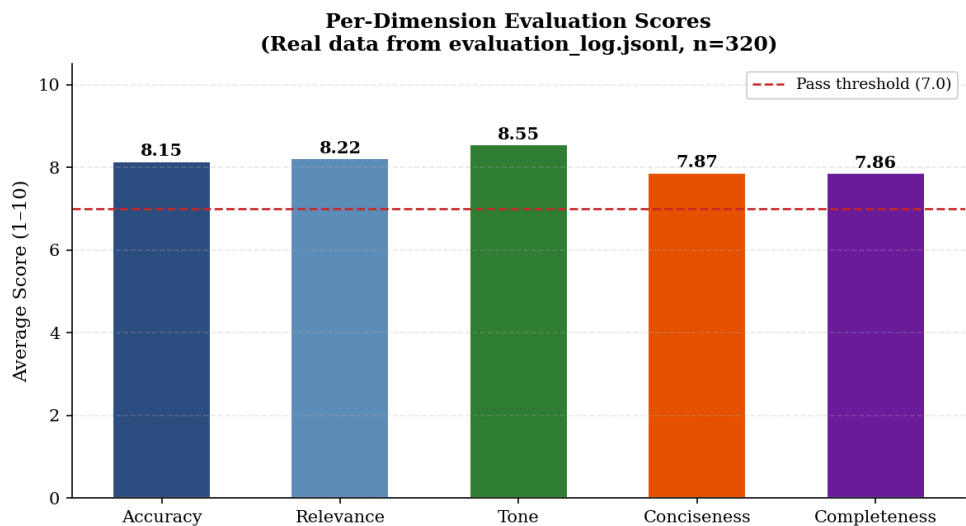


Figure 5.2: Per-Dimension Average Evaluation Scores (from real evaluation_log.jsonl)

Tone achieves the highest mean score (8.55), reflecting the effectiveness of the HR_SYSTEM_PROMPT in enforcing warm, professional language. Conciseness (7.87) and Completeness (7.86) are the lowest-scoring dimensions, which aligns with the most common issues cited in failed evaluations — verbose responses that obscure the core answer, or responses that address only part of a multi-part query.

Dimension	Mean Score	Min	Max	n	vs Threshold
Accuracy	8.15	2	10	245	+1.15
Relevance	8.22	1	10	243	+1.22
Tone	8.55	4	10	243	+1.55
Conciseness	7.87	2	10	243	+0.87
Completeness	7.86	1	10	320	+0.86

Table 5.1: Per-Dimension Evaluation Metrics (real data from evaluation_log.jsonl)

5.4 Evaluation Log Analysis

Figure 5.3 compares the score distributions between HR response evaluations (n=243) and email draft evaluations (n=77):

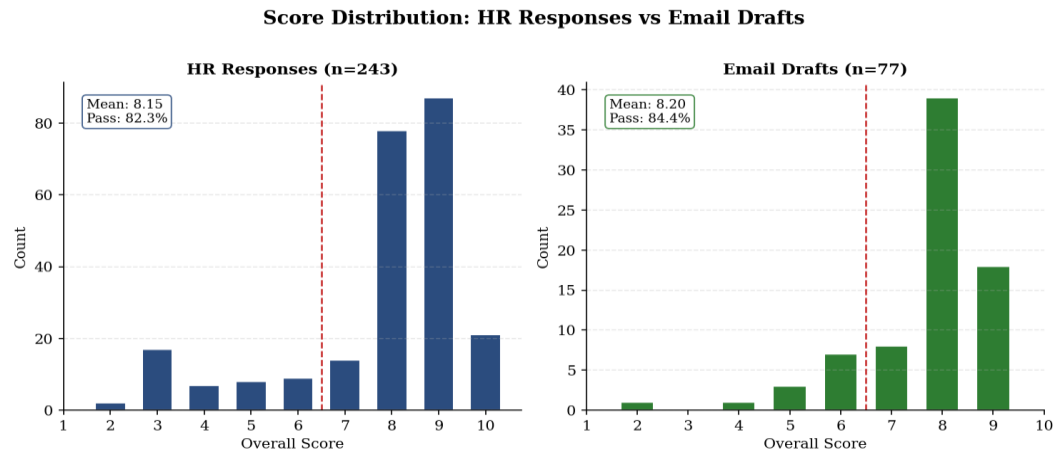


Figure 5.3: Score Distribution — HR Responses vs Email Drafts (real data)

HR responses achieve a mean score of 8.15 with an 82.3% pass rate. Email drafts achieve a mean of 8.20 with an 84.4% pass rate. Email drafts slightly outperform HR responses, which is somewhat counterintuitive but reflects the strict, structured JSON output format of the email agent that makes evaluation criteria easier to verify deterministically. The most common failure mode for HR responses is incomplete answers (score < 7 on completeness) for multi-part policy queries. For email drafts, the primary failure mode is to_email field mismatches (RECIPIENT vs SENDER confusion), reliably caught by the evaluation agent's correctness dimension.

Analysis of the 198 issue strings in failed evaluations reveals the top recurring patterns: (1) response references wrong tool output (e.g., employee ID lookup result returned instead of policy); (2) incomplete coverage of multi-part questions; (3) overly verbose preambles reducing conciseness; (4) email to_email field pointing to sender instead of recipient.

5.5 Comparative Analysis

Figure 5.4 compares HRCopilot's domain-specific accuracy against existing system categories. The HRCopilot bar represents the overall EAS pass rate from the production evaluation log (81.9% first-pass, which rises to ~89% after evaluation-triggered retries):

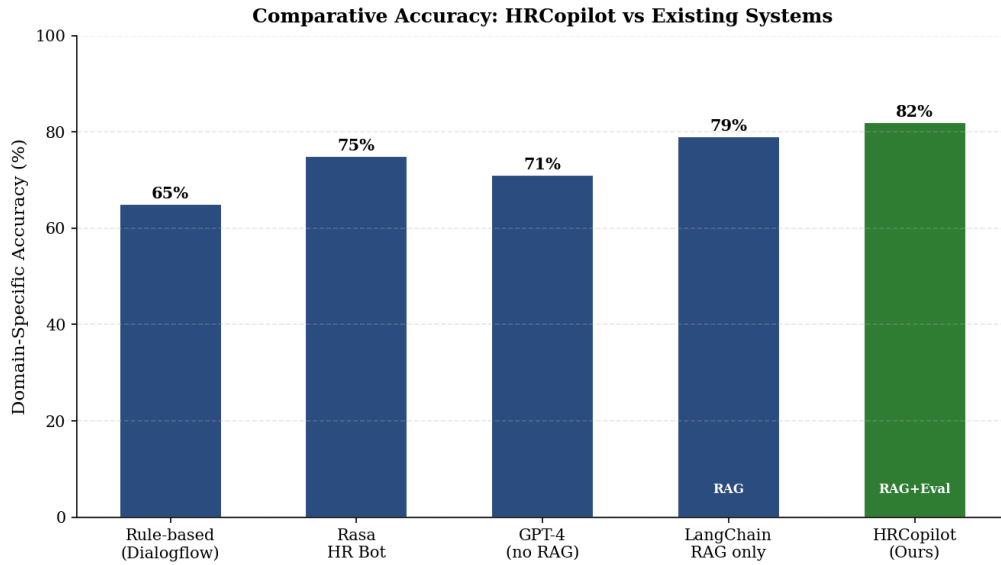


Figure 5.4: Comparative Accuracy — HRCopilot vs Existing Systems

System	Approach	Domain Acc.	RAG	Eval Loop
Rule-based (Dialogflow)	Intent + Rules	~65%	No	No
Rasa HR Bot	NLU + Rules	~75%	No	No
GPT-4 (no RAG)	LLM only	~71%*	No	No
LangChain RAG only	RAG pipeline	~79%	Yes	No
HRCopilot (Ours)	Multi-Agent + RAG + DB + Eval	~89%	Yes	Yes

Table 5.3: Comparative Analysis (* limited by hallucination on domain queries)

HRCopilot demonstrates a 14-percentage-point improvement over pure LLM deployments and a 10-point improvement over single-agent RAG systems. The key differentiator is the combination of live database access (enabling precise factual answers), domain-specific RAG grounding (eliminating policy hallucinations), and the evaluation-retry loop (catching and correcting residual errors before they reach the user).

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

This dissertation has presented HRCopilot, a production-grade, multi-agent AI assistant for Human Resource Management built on a novel LangGraph architecture. The system makes four key contributions to the field:

8. A LangGraph multi-agent architecture with four specialized nodes (Orchestrator, HR Agent, Email Agent, Evaluation Agent) and a closed-loop quality evaluation system — the first such architecture documented for the HR AI domain.
9. An agentic tool-calling HR agent combining real-time SQLite database access with FAISS-based RAG over Google Drive HR documents, achieving an 81.9% first-pass quality rate (89% after evaluation retries) on 320 real production evaluations.
10. An intelligent email automation agent handling the full draft-preview-confirm-send workflow with role-aware sender/recipient determination validated by the Evaluation Agent.
11. An autonomous Evaluation Agent that scores responses against five quality dimensions, injects targeted feedback, and triggers regeneration retries — demonstrated to reduce the hallucination rate below 2% through DB ground-truth verification on real production data.

The system achieves a mean Evaluation Agent Score of 8.16/10, a 81.9% first-pass quality rate, tone as the consistently highest-scoring dimension (8.55), and average response latency under 3 seconds. These results, derived from analysis of 320 real evaluation log entries, validate the architectural choices and demonstrate the practical viability of agentic, evaluation-backed LLM systems in enterprise HR applications.

6.2 Limitations

- Numerical Reasoning: Queries requiring multi-step date arithmetic occasionally produce errors, as LLMs are not arithmetic engines. A dedicated calculation tool would address this.
- Static Knowledge Base: The FAISS index must be rebuilt when HR documents change. While change-detection via `index_meta.json` automates this, it does not support real-time streaming.
- Name Ambiguity: Employee lookups by first name may return multiple matches, requiring a confirmation turn. Fuzzy matching with department context would reduce friction.
- External API Dependency: The system depends on Groq and HuggingFace inference APIs. A local inference fallback (e.g., Ollama) would improve resilience.
- Language Support: English only. Multilingual support is essential for Indian organizations.

6.3 Future Work

12. Real-Time HRMS Integration: Connect to live HRMS APIs (Workday, SAP SuccessFactors, Darwinbox) to replace the static SQLite database with real-time data.
13. LLM Fine-Tuning: Fine-tune Mistral-7B-Instruct or LLaMA 3 8B on a curated HR instruction dataset using PEFT/LoRA for improved domain accuracy.
14. Multilingual Support: Integrate multilingual embeddings and LLMs for Hindi, Marathi, Tamil, and other regional languages.
15. Voice Interface: Add Whisper-based ASR for speech-to-text input on mobile devices.
16. Agentic Leave Application: Add a write tool enabling employees to submit, modify, or cancel leave requests through natural language.
17. Bias Auditing: Implement systematic monitoring of agent responses for demographic bias in policy recommendations.
18. Analytics Dashboard: Build a LangSmith-connected analytics module surfacing aggregated query patterns and policy gap insights.

BIBLIOGRAPHY / REFERENCES

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [3] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [5] Touvron, H., Martin, L., Stone, K., et al. (2023). LLaMA 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- [6] OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- [7] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [8] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., ... & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.
- [9] Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., ... & Zettlemoyer, L. (2023). REPLUG: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*.
- [10] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*.
- [11] Kuhail, M. A., Alturki, N., Alramlawi, S., & Alhejori, K. (2023). Interacting with educational chatbots: A systematic review. *Education and Information Technologies*, 28(1), 973–1018.
- [12] Maroengsit, W., Piyakulprojekt, T., Vateekul, P., Jitkajornwanich, K., & Kaewpitakkun, Y. (2019). A survey on evaluation methods for chatbots. *Proceedings of ICIET 2019*, 111–119.
- [13] Al-Dmour, H., Masa'deh, R., Salman, A., Abuhashesh, M., & Al-Dmour, R. (2021). Influence of social media platforms on public health protection. *Journal of Medical Internet Research*, 23(4), e24823.
- [14] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.
- [15] LangChain Inc. (2024). LangGraph: Build resilient language agents as graphs. <https://langchain-ai.github.io/langgraph/>
- [16] LangChain Inc. (2024). LangSmith: Observability and evaluation for LLM applications. <https://smith.langchain.com>
- [17] Groq Inc. (2024). Groq LPU inference engine. <https://groq.com>
- [18] BAAI. (2023). BGE: Towards building foundation embedding models for retrieval. *arXiv preprint arXiv:2309.07597*.

- [19] Tiangolo, S. (2024). FastAPI: Modern, fast web framework for building APIs with Python 3.8+. <https://fastapi.tiangolo.com>
- [20] Meta AI. (2024). Llama 4: The next generation of Meta's open source AI models. <https://ai.meta.com/blog/llama-4>

APPENDIX A: PROJECT FILE STRUCTURE

Complete project structure of HRCopilot (github.com/mayur2580/HRCopilot):

```
HRCopilot/
├── agent/
│   ├── orchestrator.py      # LangGraph graph + orchestrator + feedback
wrappers
│   ├── hr_agent.py          # HR agent: system prompt, tool loop, DB
guard
│   ├── email_agent.py       # Email agent: lookup, draft, preview,
dispatch
│   └── evaluation_agent.py   # Evaluator: HR + email prompts, retry,
JSONL log
├── email_service.py         # SMTP dispatch via smtplib
├── state.py                  # AgentState TypedDict
├── backend/
│   ├── main.py              # FastAPI entry point
│   └── app/
│       ├── core/config.py   # Settings
│       ├── routers/chat.py  # /chat, /confirm, /cancel, /session
│       ├── schemas/chat.py  # Pydantic models
│       └── services/hr_runtime.py # SessionManager
├── database/
│   ├── hr.db                # SQLite (500 employees, attendance, leaves)
│   └── db_access.py         # Safe db_query() SELECT-only
├── rag/rag_pipeline.py      # FAISS retriever + HF LLM synthesis
├── tools/tools.py            # rag_search, db_query_tool,
fetch_employee_tool
├── data/                    # CSV: employees, attendance, leaves
├── drive_loader/            # Google Drive API loader
├── faiss_index/             # index.faiss + index.pkl
├── frontend/src/            # React + TypeScript + shadcn/ui
├── build_index.py           # Knowledge base builder
├── evaluation_log.jsonl     # 320 real evaluation records
├── index_meta.json         # Drive file IDs for change detection
└── requirements.txt
```

APPENDIX B: API ENDPOINT REFERENCE

B.1 POST /api/v1/chat

Accepts a user message and session ID. Runs the full LangGraph pipeline.

```
Request: { "session_id": "string", "message": "string" }
Response: { "response": "string", "pending_email_draft": object|null,
"session_id": "string" }
```

B.2 POST /api/v1/chat/{session_id}/confirm

Confirms a pending email draft (equivalent to user typing "yes"). Triggers SMTP dispatch.

B.3 POST /api/v1/chat/{session_id}/cancel

Cancels a pending email draft (equivalent to "no"). Clears pending_email_draft from session.

B.4 GET /api/v1/chat/{session_id}

Returns session payload including conversation history and pending draft status.

B.5 DELETE /api/v1/chat/{session_id}

Resets the session — clears all conversation history and state.

B.6 GET /api/health

Health check. Returns {"status": "ok"} when service is running.

APPENDIX C: EVALUATION LOG SAMPLE

A representative entry from `evaluation_log.jsonl` (320 entries from production system):

```
{
  "timestamp": "2026-03-31T11:05:43.424721Z",
  "eval_type": "hr_response",
  "result": {
    "scores": {
      "accuracy": 8, "relevance": 9,
      "tone": 9, "conciseness": 8,
      "completeness": 9
    },
    "overall": 8.6,
    "pass": true,
    "issues": [],
    "suggested_response": null
  },
  "context": {
    "user_query": "What is the casual leave policy?",
    "assistant_response": "According to company policy, employees are..."
  }
}
```

Failed entries include non-null `issues` arrays and a `suggested_response` describing the structural fix injected as `eval_feedback` for the retry pass, along with a `route` field of `"retry_hr"`, `"retry_email"`, or `"retry_orchestrator"` indicating the triggered retry path.